T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 7/24/26 | **DATE SUBMITTED:** | 29-7-26 |
| **NAME:** | Condino, Niel Vincent B. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 3** **Linked Lists**

## 1. PROCEDURE OUTPUT

**Screenshot**

```
15  void runSLCode(){        //Creation of node in the stack
16      SingleList<char>* head = nullptr;
17      SingleList<char>* sec = nullptr;
18      SingleList<char>* thrd = nullptr;
19      SingleList<char>* frth = nullptr;
20      SingleList<char>* ffth = nullptr;
21      SingleList<char>* last = nullptr;
22
23      //allocates memory in the heap
24      head = new SingleList<char>;
25      sec = new SingleList<char>;
26      thrd = new SingleList<char>;
27      frth = new SingleList<char>;
28      ffth = new SingleList<char>;
29      last = new SingleList<char>;
30
31
32      //Initializing of the data in the list
33      head->data = 'C';
34      sec->data = 'P';
35      thrd->data = 'E';
36      frth->data = '0';
37      ffth->data = '1';
38      last->data = '0';
39
40      //Link the list
41      head->next = sec;
42      sec->next = thrd;
43      thrd->next = frth;
44      frth->next = ffth;
45      ffth->next = last;
```

```
❯ cd "/home/nielc/Downloads/HOA/HW #3.1/" && g++ main.cpp -o main && "/home/nielc/Downloads/HOA/HW #3.1/"main

~/Downloads/HOA/HW #3.1
```

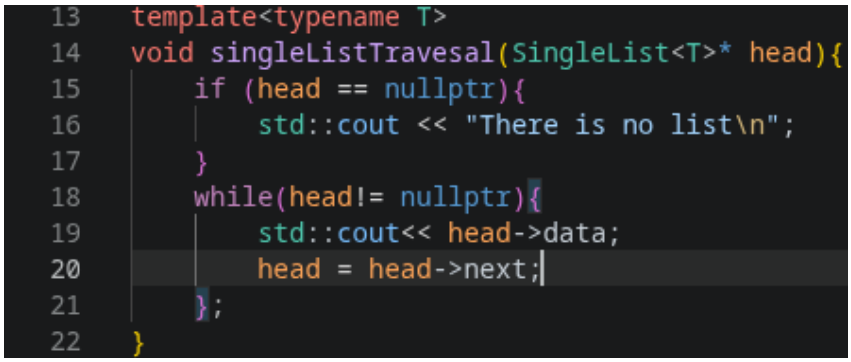
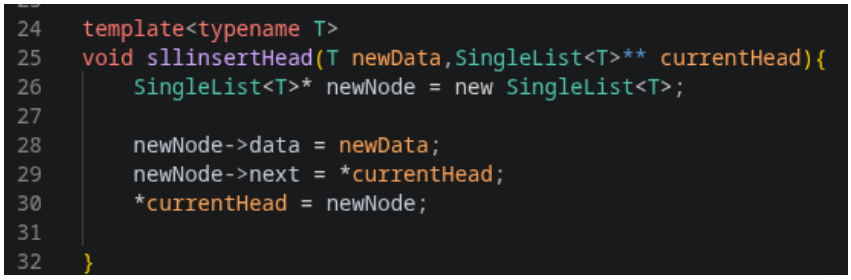
**Discussion**

This is a pretty simple implementation of a single linked list. It first creates the element's pointer. It then allocates memory for the instances, after which their data is stored. Lastly, all of them are linked together. In its output, it shows nothing because

| | |
|---|---|
| | no printing command is called, but the instances are stored in memory. There are no ways to traverse, insert, or delete the nodes easily. |

**Table 3-1.Output of Initial/Simple implementation**

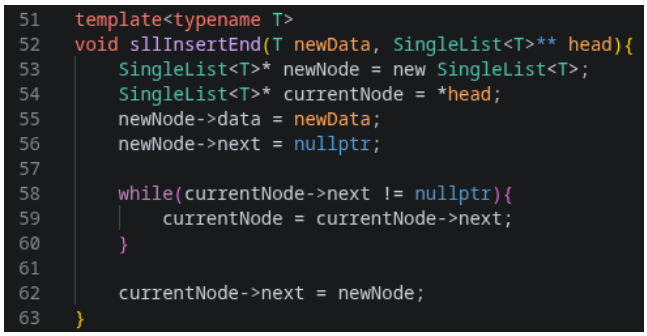
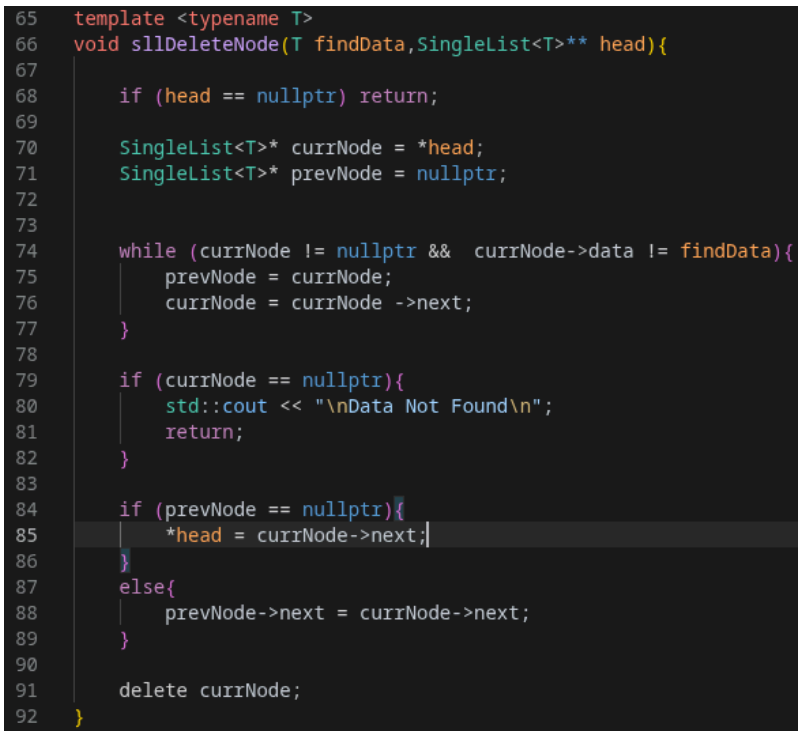
The code above is a simple implementation of the linked list. The linked list for this program is created using a class with members for data and the pointer to where the node is placed in memory. In the program, it creates the list elements, sets their data, and links them together using their pointers. This part of the activity is just the start of the procedure in creating linked lists, as there are still functionalities missing from it. These functionalities will be tackled and implemented throughout the procedure.

| Operation | Screenshot |
|---|---|
| Traversal |  |
| Insertion at head |  |
| Insertion at any part of the list |  |

```
13  template<typename T>
14  void singleListTravesal(SingleList<T>* head){
15      if (head == nullptr){
16          std::cout << "There is no list\n";
17      }
18      while(head!= nullptr){
19          std::cout<< head->data;
20          head = head->next;
21      };
22  }
```

```
24  template<typename T>
25  void sllinsertHead(T newData,SingleList<T>** currentHead){
26      SingleList<T>* newNode = new SingleList<T>;
27
28      newNode->data = newData;
29      newNode->next = *currentHead;
30      *currentHead = newNode;
31
32  }
```

```
34  template <typename T>
35  void sllGeneralInsert(T newData,SingleList<T>* prevNode){
36      //If check if head
37      if (prevNode == nullptr){
38          std::cout << "Previous value cannot be null\n";
39          return;
40      }
41      //Allocate new node
42      SingleList<T>* newNode = new SingleList<T>;
43      //Store Data
44      newNode->data = newData;
45      //Point new node
46      newNode->next = prevNode->next;
47      prevNode->next = newNode;
48
49  }
```

| | |
|---|---|
| Insertion at the end |  |
| Deletion of a node |  |

```
51  template<typename T>
52  void sllInsertEnd(T newData, SingleList<T>** head){
53      SingleList<T>* newNode = new SingleList<T>;
54      SingleList<T>* currentNode = *head;
55      newNode->data = newData;
56      newNode->next = nullptr;
57
58      while(currentNode->next != nullptr){
59          currentNode = currentNode->next;
60      }
61
62      currentNode->next = newNode;
63  }
```

```
65  template <typename T>
66  void sllDeleteNode(T findData,SingleList<T>** head){
67
68      if (head == nullptr) return;
69
70      SingleList<T>* currNode = *head;
71      SingleList<T>* prevNode = nullptr;
72
73
74      while (currNode != nullptr &&  currNode->data != findData){
75          prevNode = currNode;
76          currNode = currNode ->next;
77      }
78
79      if (currNode == nullptr){
80          std::cout << "\nData Not Found\n";
81          return;
82      }
83
84      if (prevNode == nullptr){
85          *head = currNode->next;
86      }
87      else{
88          prevNode->next = currNode->next;
89      }
90
91      delete currNode;
92  }
```

**Table 3-2.Code for the List Operation**

The code above shows the different operations for the singly linked list. List traversal is created by using the algorithm of printing the data of the currentNode, then going to the next. This is done in a loop that stops whenever the current node does not exist. There are three different methods of inserting a node inside the list. First, there is also a function for creating a new head of the list, which works by creating a new node and pointing the given head of the list to the newly created node. Next, there is a function for inserting a node somewhere in the middle of the list. This works by getting a node that will be placed before the new node, pointing this node to the newly created node, then pointing the new node to the last node's "next" pointer before the insertion. The last function inserts a node at the end of the list. This works by traversing the end of the list, creating a new node, and pointing the last node to it. Then we have the last two functions, which delete a node. It first finds the node to be deleted by matching a node's data to the given value. If a node is found, it links the one before it and after it together by changing the before node's next pointer to point to the next of the found node. Lastly, this function deletes the found node.

| | | |
|---|---|---|
| a | **Source Code** | `//Prints all the element of the list`<br>`std::cout << "Testing travesal\n";`<br>`singleListTravesal(head);`<br>`std::cout << std::endl;` |
| a | **Console** | `Testing travesal`<br>`CPE101` |
| b | **Source Code** | `//Inserts a new head of the list`<br>`std::cout << "Testing of sllinsertHead\n";`<br>`sllinsertHead(newData: 'G', currentHead: &head);`<br>`singleListTravesal(head);`<br>`std::cout << std::endl;` |
| b | **Console** | `Testing of sllinsertHead`<br>`GCPE101` |
| c | **Source Code** | `//Inserts a data somewhere in the middle of the list`<br>`std::cout << "Testing of sllGeneralInsert\n";`<br>`sllGeneralInsert(newData: 'E', prevNode: head->next->next);`<br>`singleListTravesal(head);`<br>`std::cout << std::endl;` |
| c | **Console** | `Testing of sllGeneralInsert`<br>`GCPEE101` |
| d | **Source Code** | `//Deleting the node with the data "C"`<br>`std::cout << "Testing deleting node with C\n";`<br>`sllDeleteNode(findData: 'C', head: &head);`<br>`singleListTravesal(head);`<br>`std::cout << std::endl;` |
| d | **Console** | `Testing deleting node with C`<br>`GPEE101` |
| e | **Source Code** | `//Deleting the node with the data "P"`<br>`std::cout << "Testing deleting node with P\n";`<br>`sllDeleteNode(findData: 'P', head: &head);`<br>`singleListTravesal(head);`<br>`std::cout << std::endl;` |

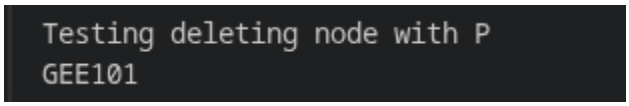
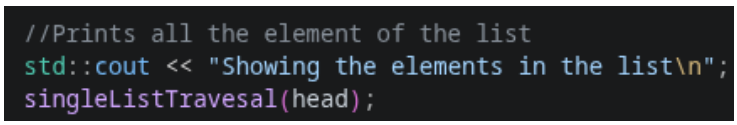
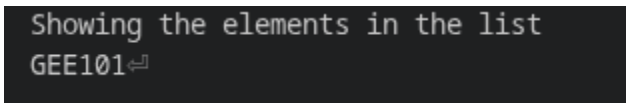
| | | |
|---|---|---|
| e | **Console** |  |
| f | **Source Code** |  |
| f | **Console** |  |

Table 3-3. Code and Analysis for Singly Linked Lists

In section a, the singlelist traversal function, which was already explained from the previous figure, was tested. Given the created linked lists, it behaves as it should, as it starts from the head, which contains the “C” data. Then stops at the last element, which will be null. Next, in section b, it tests the function that inserts a node as a new head. For demonstration, it inserts a node with a “G” as the new head. After inserting it, it reprints the linked list, and as expected, it prints out the G at the start of CPE101. Next, it inserts a node with an “E” for its data next to the letter P. It invokes the function, and as seen in the output, it places an E next to the first P. The next sections d and e delete nodes containing C and P data in them. This is again reflected when traversing the list. Then section f shows the newly created word, which is GEE101, but inside it looks like G -> E -> E -> 1 -> 0 -> 1 -> null from C -> P -> E -> 1 -> 0 -> 1. This linked list is modified using the different functions shown in Figure 2.

| Screenshot(s) | Analysis |
|---|---|
|  | The forward traversal of the list is the same as in a single linked list. It uses a loop that traverses each element of the list. However, a tail variable is used to track the last node checked. This tail is used for the backwards traversal. Using this tail, it traverses using the prev pointer variable to go to the previous member of the list. Because of the loops that check each element of the list, this function has a runtime of O(n). |

```
25  template <typename T>
26  void dllTraverse(DoubleList<T>* currentNode){
27      if (currentNode == nullptr){
28          std::cout << "List is empty\n";
29          return;
30      }
31
32      DoubleList<T>* tail;
33      std::cout << "Forward\n";
34
35      while (currentNode != nullptr){
36          std::cout << currentNode->data<< " -> ";
37          tail = currentNode;
38          currentNode = currentNode->next;
39      }
40
41      std::cout << "null";
42      std::cout << "\n";
43      std::cout << "Backward\n";
44
45      while (tail != nullptr){
46          std::cout << tail->data<< " -> ";
47          tail = tail->prev;
48      }
49
50      std::cout << "null";
51      std::cout << "\n";
52  }
```

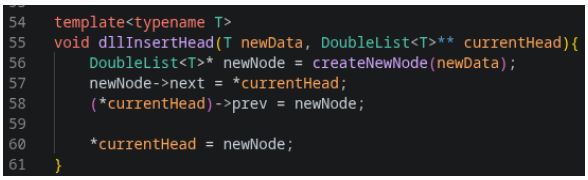
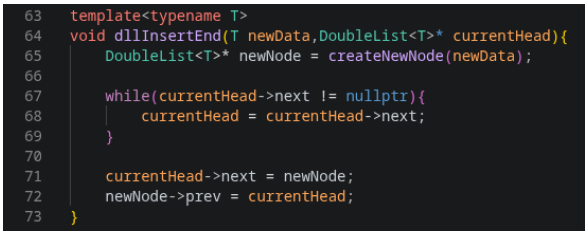
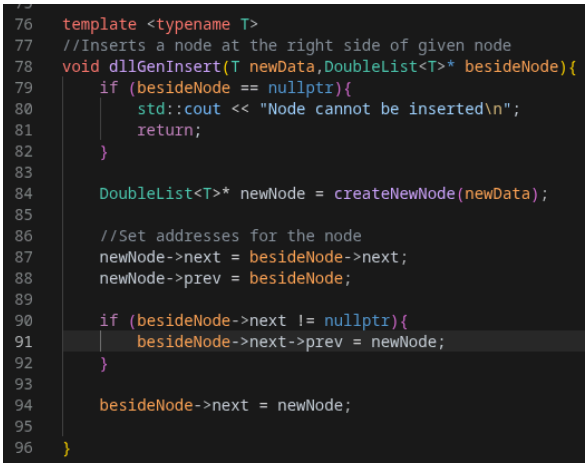
| | |
|---|---|
|  | This code shares the logic as the one on the singly linked list. The only difference is that it connects the currentHead's prev pointer to the new node. This has a time complexity of O(1). |
|  | This code shares the logic as the one on the singly linked list. The only difference is that it connects the current node's next pointer to the new node. This has a time complexity of O(n) because of the loop. |
|  | This one has its logic changed more by being a doubly linked list. It inserts the new node next to the given node. It then sets the "next" pointer to point to where the given node's next points to and the prev points to the given node. An error check is then implemented to make sure the next node of the given node is not null. Then, if it is not null, the next node's prev pointer is pointed to the new node. This function has an O(1) time complexity. |
|  | This logic is mostly the same as on the singly linked list. The different part is the fixing of links at the last part. The next node from the to-be-deleted node's prev member is linked to the to-be-deleted node'prev. The node before the to-be-deleted node's next member is set to the to-be-deleted node's next. After that, the same node is deleted. This function has time complexity of O(n) due to the loop that finds the to-be-deleted node. |

Table 3-4. Modified Operations for Doubly Linked Lists

The table above shows the modifications made to the singly linked list methods to modify or print the list to make it work with doubly linked lists. The analysis is provided in the table on how they work and their time complexity.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
15  int main(){
16      appStart();
17      return 0;
18  }
19
20  void appStart(){
21      Menu mainMenu = Menu(givenMenuSize: 2,menuTitleGiven: "IPlayMusic");
22      mainMenu.EditMenu(index: 0, menuDescription: "Playlist 1");
23      mainMenu.EditMenu(index: 1,menuDescription: "Exit");
24
25      DoubleList<std::string>* playlistHead = nullptr;
26      //DoubleList<std::string>* originalHead = playlistHead;
27
28      int menuChoice = mainMenu.NavigateMenu();
29
30      if (menuChoice == 0){
31          appMain(headPtr: &playlistHead);
32      }
33
34      clearScreen();
35
36      std::cout << "App exited";
37
38      dllDeleteList(head: &playlistHead);
39  }
```

**Figure 1: Driver code**

This code is used to start the menu of the program. It first creates a menu object and sets the menu's choices. It first creates a doubly linked list for the head of the list. It then lets the user choose from the menu. If the user chooses to enter the program, it invokes the appMain() function. After the user is done with the app, it clears the screen, prints out a closing message, and deletes the list to avoid memory leakage.

```
====================================
         IPlayMusic
====================================
 > Playlist 1
   Exit

Use UP/DOWN arrows to navigate. Press ENTER to select.
```

**Figure 2: Main menu**

The figure above shows the main menu of the program. This menu can be navigated using the arrow keys or the mouse scroll. The user can either proceed with the program by choosing playlist 1 or exit the program by choosing that option.

```
41  void appMain(DoubleList<std::string>** headPtr){
42      bool hasExited = false;
43
44      clearScreen();
45
46      while (!hasExited){
47              //Insert first song
48          if (*headPtr == nullptr){
49              std::string songName = getNoneEmptyString(message: "Please enter first song name:")
50
51              *headPtr = createNewNode(newData: songName);
52
53          }
54          if (actionMenu(headPtr) == 0){
55              hasExited = true;
56          }
57      }
58  }
```

**Figure 2: AppMain code**

This initializes the program properly. It creates a bool for the loop of the program, clears the screen, then loops until the app is exited by the user. If there is no data for the head ptr, it gets input from the user for the song name and creates a node for the head of the list with the input from the user. It then calls the actionMenu() function, which returns an int. If this function returns 0, the exit signal, the loop will end.

```
Please enter first song name:Song 1
```

```
Please enter first song name:
Input cannot be empty
```

**Figure 3: Input first song output**

This figure shows how the user can enter a song name for the program. This function does not accept any empty string so that there is no empty string entered by the user.

```cpp
int actionMenu(DoubleList<std::string>** headPtr){
    Menu actions = Menu(givenMenuSize: 5,menuTitleGiven: "Choose Action");
    actions.EditMenu(index: 0, menuDescription: "Insert more songs");
    actions.EditMenu(index: 1,menuDescription: "Delete songs");
    actions.EditMenu(index: 2,menuDescription: "Play songs");
    actions.EditMenu(index: 3,menuDescription: "Loop Songs");
    actions.EditMenu(index: 4, menuDescription: "Go back");

    int choice = actions.NavigateMenu();

    switch(choice){
        case 0:{
            insertSongs(headPtr);
            break;
        }
        case 1:{
            clearScreen();
            std::string toBeRemoved = getNoneEmptyString(message: "Enter the name of the song you want to remove:");
            dllDeleteNode(findData: toBeRemoved, head: headPtr,err_mess: "Song Not Found\n",succ_mess: "Song successfully deleted")
            waitS(time: 2);
            clearScreen();
            break;
        }
        case 2:{
            playSongsOneByOne(headPtr);
            break;
        }
        case 3:{
            playLoopedSongs(headPtr: *headPtr);
            break;
        }
        case 4:{
            return 0;
        }
    }
    return 1;
}
```

**Figure 3:actionMenu**

This function serves as the central control loop for the program by initializing and managing a menu driven by the Menu class. It prompts the user with five primary operations: inserting, deleting, navigating, and looping songs, and executes a function based on the chosen integer choice. Memory modification of the playlist structure is enabled by passing the double pointer headPtr directly into underlying helper functions. The switch block handles each operational routine while maintaining clean console feedback through explicit screen clears and pause delays. Finally, it returns an integer status code, passing 1 to sustain the continuous application loop or 0 when the user signals to navigate backward.

```
====================================
      Choose Action
====================================
 > Insert more songs
   Delete songs
   Play songs
   Loop Songs
   Go back

Use UP/DOWN arrows to navigate. Press ENTER to select.
```

**Figure 4:Action menu**

In this menu, the user can interact with the interface and pick from the given options on what to do. The actions are as presented: insert, delete, play, and loop songs. Each of the options provides a menu for their chosen actions. The user can also go back to the main menu to exit the program.

```
 98  void insertSongs(DoubleList<std::string>** headPtr){
 99      clearScreen();
100      Menu insertSongMenu = Menu(givenMenuSize: 3,menuTitleGiven: "Where to insert song");
101      insertSongMenu.EditMenu(index: 0, menuDescription: "Start");
102      insertSongMenu.EditMenu(index: 1, menuDescription: "Pick a place");
103      insertSongMenu.EditMenu(index: 2, menuDescription: "End");
104
105      int choice = insertSongMenu.NavigateMenu();
106
107      std::string songName = getNoneEmptyString(message: "Enter song name:");
108
109      switch (choice) {
110          case 0:
111          {
112              dllInsertHead(newData: songName,currentHead: headPtr);
113              break;
114          }
115          case 1:{
116              DoubleList<std::string>* foundNode = nullptr;
117              int tries = 3;
118              do{
119                  std::string besideSongName = getNoneEmptyString(message: "Put next to what song:");
120
121                  foundNode = dllFindValue(currentNode: *headPtr, dataNeeded: besideSongName);
122
123                  clearScreen();
124                  //Error correction
125                  if (foundNode == nullptr) std::cout << "Song not found, please retype\n";
126                  std::cout << "Tries left " << --tries << '\n';
127
128              } while (foundNode == nullptr && tries > 0);
129              clearScreen();
130              dllGenInsert(newData: songName, besideNode: foundNode);
131              break;
132          }
133          case 2:{
134              dllInsertEnd(newData: songName, currentHead: *headPtr);
135          }
136      }
137  }
```

**Figure 5:insertSongs**

This function manages the addition of new nodes to the doubly linked list by presenting a submenu with options to insert at the beginning, middle, or end. After collecting the target song title from standard input using getNonEmptyString, it routes execution to a specific pointer manipulation function for the selected insertion context. For intermediate insertions, it incorporates a retry loop allowing up to three attempts to locate a reference target node using dllFindValue. If the search succeeds, it delegates the pointer reassignment to dllGenInsert, whereas failure triggers a diagnostic prompt and gracefully degrades without corrupting the structure. Ultimately, the routine abstracts positional insertion mechanics from the rest of the application flow.

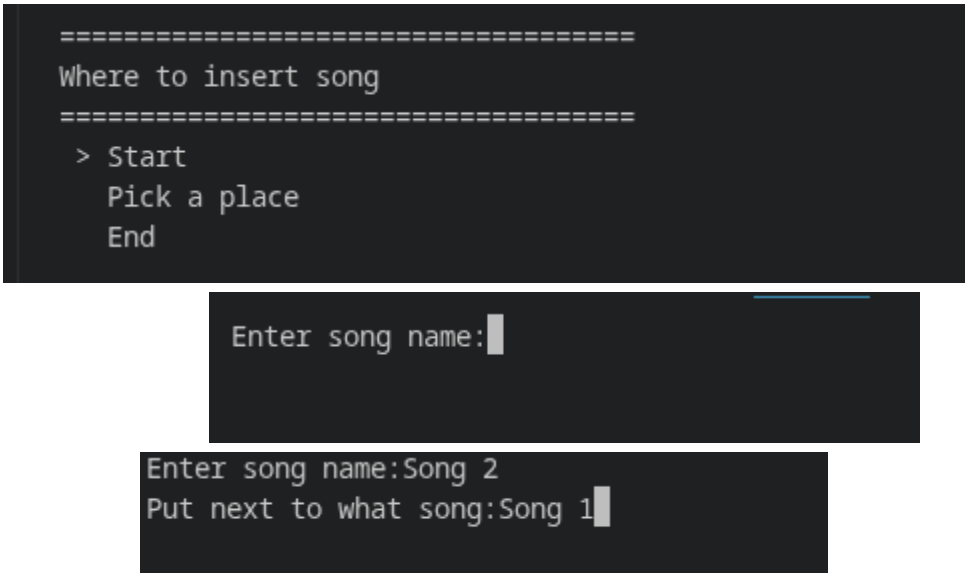



**Figure 6:Song insertion**

This menu lets the user choose where they will put their songs. This will let the user enter the name of the song based on the given input from the user. When choosing the “pick a place” option, it will let the user enter the name of the song they want it to play after.

```
139  void playSongsOneByOne(DoubleList<std::string>** headPtr){
140      DoubleList<std::string>* currentNode = *headPtr;
141
142      if (currentNode == nullptr) return;
143
144      while (currentNode != nullptr) {
145          if (!songActionMenu(currentNode: &currentNode)) break;
146      }
147  }
```

```
149 int songActionMenu(DoubleList<std::string>** currentNode){
150     if ((*currentNode)->next == nullptr && (*currentNode)->prev == nullptr){
151         Menu songMenu = Menu(givenMenuSize: 1,menuTitleGiven: "Song: " + (*currentNode)->data);
152         songMenu.EditMenu(index: 0,menuDescription: "Go back");
153 
154         songMenu.NavigateMenu();
155 
156         return 0;
157     }
158     else if ((*currentNode)->next == nullptr){
159         Menu songMenu = Menu(givenMenuSize: 2,menuTitleGiven: "Song: " + (*currentNode)->data);
160         songMenu.EditMenu(index: 0,menuDescription: "Play previous song");
161         songMenu.EditMenu(index: 1,menuDescription: "Go back");
162 
163         int choice = songMenu.NavigateMenu();
164 
165         if (choice == 0){
166             (*currentNode) = (*currentNode)->prev;
167         }
168         else {
169             return 0;
170         }
171     }
172     else if ((*currentNode)->prev == nullptr){
173         Menu songMenu = Menu(givenMenuSize: 2,menuTitleGiven: "Song: " + (*currentNode)->data);
174         songMenu.EditMenu(index: 0,menuDescription: "Play next song");
175         songMenu.EditMenu(index: 1,menuDescription: "Go back");
176 
177         int choice = songMenu.NavigateMenu();
178 
179         if (choice == 0){
180             (*currentNode) = (*currentNode)->next;
181         }
182         else {
183             return 0;
184         }
185     }
186     else{
187         Menu songMenu = Menu(givenMenuSize: 3,menuTitleGiven: "Song: " + (*currentNode)->data);
188         songMenu.EditMenu(index: 0,menuDescription: "Play next song");
189         songMenu.EditMenu(index: 1,menuDescription: "Play previous song");
190         songMenu.EditMenu(index: 2,menuDescription: "Go back");
191 
192         int choice = songMenu.NavigateMenu();
193 
```

```
194         if (choice == 0){
195             (*currentNode)= (*currentNode)->next;
196         }
197         else if (choice == 1) {
198             (*currentNode) = (*currentNode)->prev;
199         }
200         else {
201             return 0;
202         }
203     }
204 
205     return 1;
206 }
```

**Figure 7:Song Playing code**

The first function, playSongOneByOne does a specific task. This function lets the user step through and play songs in order starting from the head. It uses a pointer variable to keep track of the song currently being played. Before starting the playback loop, it checks if the list is empty so the program does not crash. Inside the loop, it calls songActionMenu to show choices based on the current song. The loop stops playing as soon as the menu returns a value of 0. Next is the songActionMenu() . This function builds a simple menu depending on where the current song is inside the list. It checks if the song has previous or next songs, so it only shows choices that actually make sense. For example, a song at the very end of the list will only show options to go backward or exit. When the user picks a direction, it changes the node pointer to move forward or backward. It returns 1 if the user moves to another song, or 0 if they choose to exit.

```
====================================
       Song: Song 0
====================================
 > Play next song
   Go back

Use UP/DOWN arrows to navigate. Press ENTER to select.
```

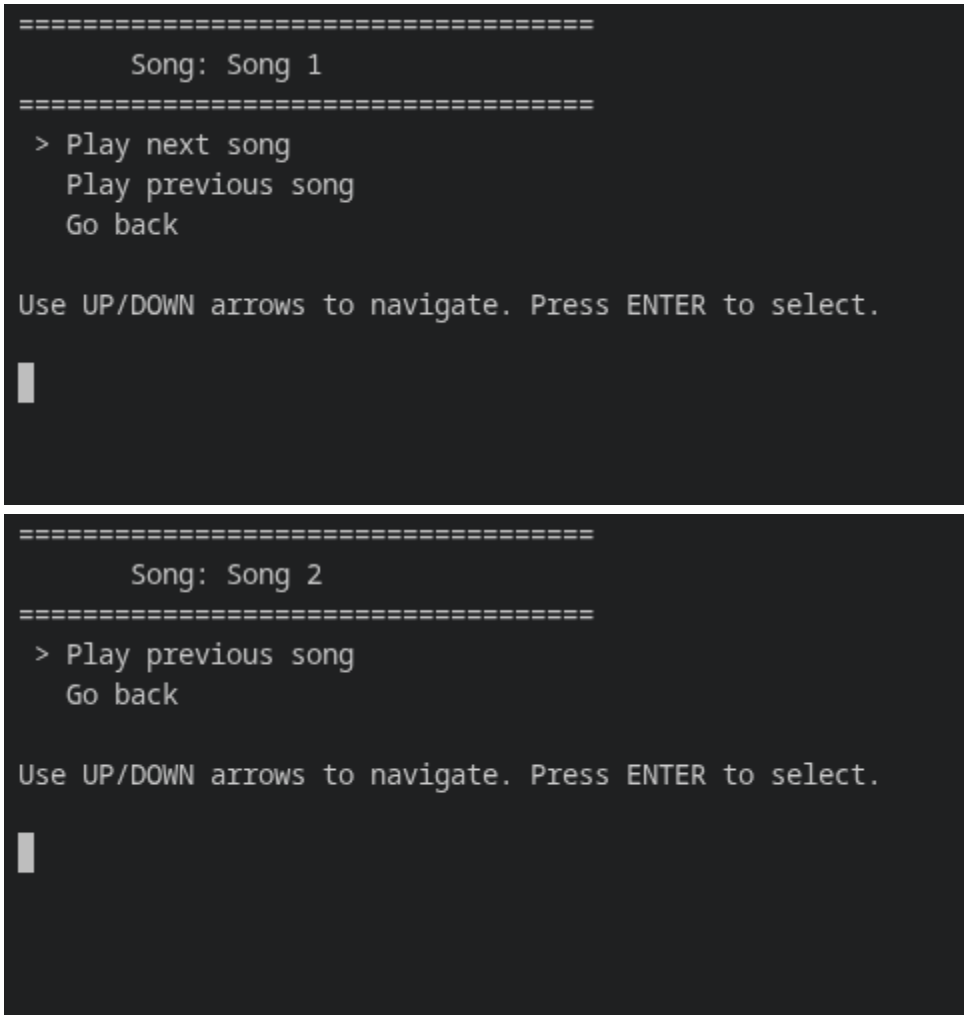



**Figure 8:Play songs output**

This figure shows the menu when playing the songs. If the song is the head of the list, then it just shows the play next music option and the go back option. If it is the last song, you can only choose the previous song and go back option. Lastly, if it does not qualify for those conditions above, the user can either play the next or previous song, or the user can also go back to the action menu.

```cpp
void playLoopedSongs(DoubleList<std::string>* headPtr){
    clearScreen();
    DoubleList<std::string>* tail = headPtr;

    //Connect the list tail to its head to allow circular access
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    headPtr->prev = tail;
    tail->next = headPtr;

    DoubleList<std::string>* currentNode = headPtr;

    bool continueLoop = true;

    while (continueLoop){
        Menu songMenu = Menu(givenMenuSize: 3,menuTitleGiven: "Song: " + currentNode->data);
        songMenu.EditMenu(index: 0,menuDescription: "Play next song");
        songMenu.EditMenu(index: 1,menuDescription: "Play previous song");
        songMenu.EditMenu(index: 2,menuDescription: "Exit Loop");

        int choice = songMenu.NavigateMenu();

        if (choice == 0){
            currentNode= currentNode->next;
        }
        else if (choice == 1) {
            currentNode = currentNode->prev;
        }
        else {
            //Revert back to normal
            headPtr->prev = nullptr;
            tail->next = nullptr;
            continueLoop = false;
        }
    }
}
```

**Figure 9:playLoopedSongs**

This function temporarily turns the standard list into a circular list so songs can loop continuously. It finds the last node in the list and connects its next pointer back to the head node. It then shows a menu that lets the user move back and forth through the playlist without reaching an end. When the user chooses to exit, it disconnects the loop by setting the end pointers back to null. Cleaning up these pointers is essential so that other parts of the code do not get stuck in endless loops.

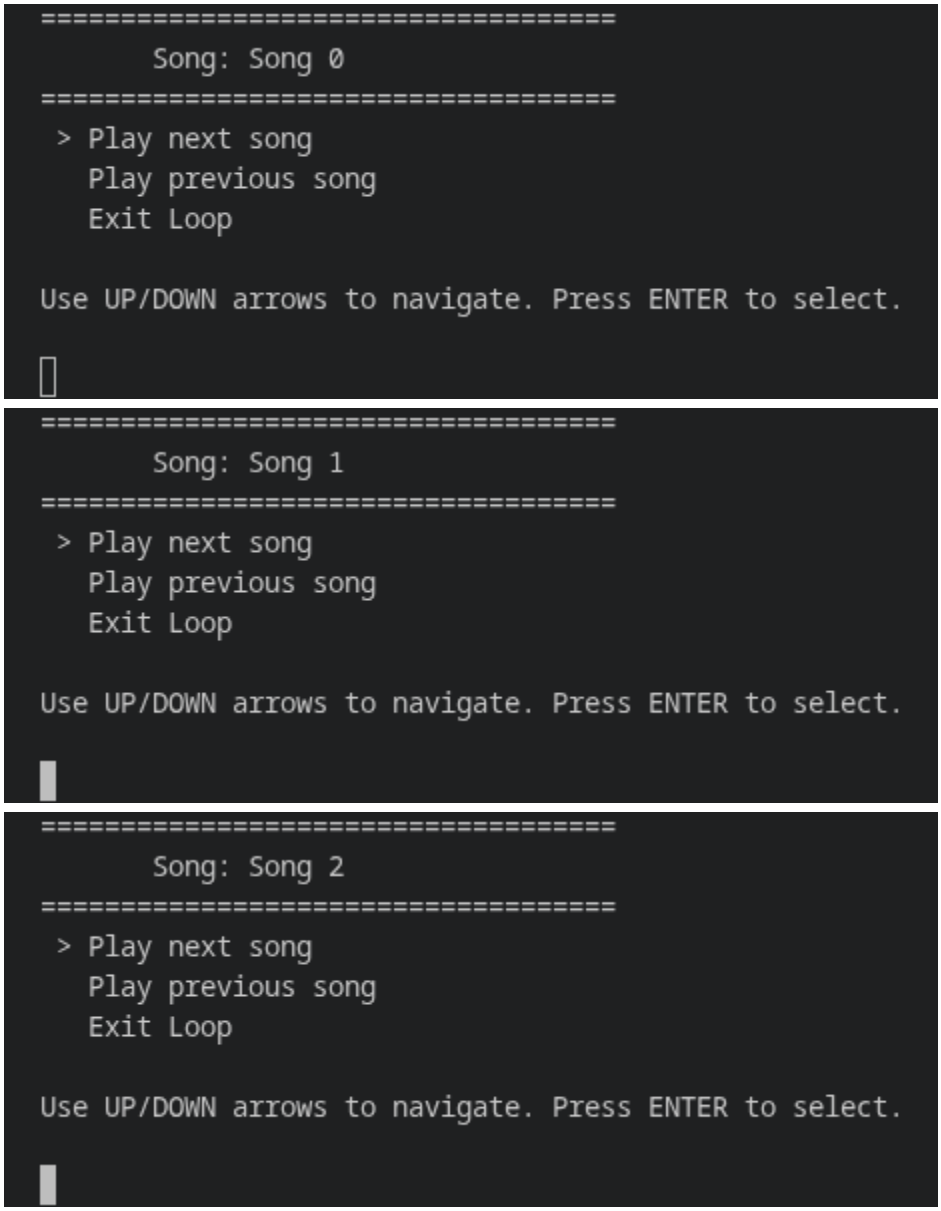



**Figure 10: Looped Song output**

This output shows how the looped playing of songs works. As in Figure 8, the songs can choose to play the next, previous, or exit the loop. Because the head and the tail are linked, the user can safely choose the next song or the previous song without worrying about a memory-accessing error when trying to read a null pointer. This means when the “tail” or “head” is reached, the next and previous song choices can still work with no errors.

**Conclusion:**

In this activity, I learned about the concept of linked lists and how they are implemented. Unlike arrays, linked list members' memory addresses are not next to each other, so they are connected or linked using a pointer. Also, unlike arrays, they don't have a fixed size, but you cannot find data using an index; instead, you need to traverse the list from the head, which can take a long time based on the number of elements. From this activity, I also learned the different methods associated with linked lists, such as traversing it, inserting an element, and deleting a node or an entire list. I also learned about the singly and doubly linked list. Doubly giving a way to go back to the previous node, as they provide a pointer to the previous element. However, doubly linked lists are also more complex than just the singly linked ones. In the supplementary activity, I was able to apply more of my understanding of this concept and able to create a circular linked list letting me to loop through the playlist needed. Overall, I learned about the concept and implementation of linked lists, and I hope I can expand my knowledge about it and know how to use it as a stack.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.